

IMDB 电影评论情感分析实验报告

方向：自然语言处理 (NLP)

任务：IMDB 电影评论情感二分类

模型：BERT-base-uncased 微调

2026 年 6 月 7 日

1 实验环境

1.1 硬件环境

项目	配置
GPU	NVIDIA GeForce RTX 5060 Laptop GPU (8 GB VRAM)
CUDA	12.8
CPU	AMD 系列处理器
RAM	DDR5 系统内存

1.2 软件环境

库 / 工具	版本
Python	3.10
PyTorch	2.12.0.dev20260408+cu128
Transformers	4.47.0
datasets	3.2.0
scikit-learn	1.7.2
matplotlib / seaborn	3.10.9 / 0.13.2
numpy	2.2.6
tqdm	4.68.1

2 数据准备与预处理

2.1 IMDB 数据集

IMDB Movie Reviews 数据集包含 50,000 条英文电影评论，每条评论带有情感标签：label=1（正面）或 label=0（负面）。数据集正负样本各 25,000 条，分布均衡，无需额外处理类别不平衡问题。

2.2 数据集划分

按照考核要求，将原始训练集按 9:1 比例随机划分为训练集和验证集：

数据集	样本数	用途
训练集	22,500（原始训练集 90%）	模型参数学习
验证集	2,500（原始训练集 10%）	监控过拟合，选择最佳模型
测试集	25,000（原始测试集全部）	最终性能评估

2.3 文本预处理

文本预处理包含以下四个步骤：

- 分词（Tokenization）：使用 bert-base-uncased 的 WordPiece 分词器，将原始文本转为 token ID 序列。分词器自动处理大小写统一（uncased 模式）和稀有词的子词拆分（如「unbelievable」→「un」+「##believable」）。
- 截断（Truncation）：设置 max_length=256，超过部分从尾部截断。IMDB 评论平均长度约 230 词，256 覆盖了大部分评论内容。
- 填充（Padding）：不足 256 词的部分补零，同时生成 attention_mask 标记有效位置（1 = 真实文本，0 = 填充）。

4. 批处理（Batching）：使用 DataLoader 按 batch_size=16 分组，训练集开启 shuffle=True 随机打乱。

3 BERT 微调实验

3.1 模型架构与超参数

- 1. 采用 HuggingFace 的 bert-base-uncased 预训练模型（约 1.1 亿参数）作为骨干网络。在 [CLS] 标记的 768 维输出向量上，添加一个全连接分类层（768 → 2），将语义向量映射为正面/负面两类的 logits 分数。
- 2. BERT 核心组件：12 层 Transformer Encoder，每层 12 个注意力头，隐藏维度 768，前馈网络中间层 3072，激活函数 GELU，Dropout 率 0.1。
- 3. 关键超参数配置如下：

超参数	取值	说明
MAX_LENGTH	256	IMDB 评论平均约 230 词，256 覆盖大部分
BATCH_SIZE	16	8 GB 显存的安全稳定值
LEARNING_RATE	2×10^{-5}	BERT 微调黄金学习率
NUM_EPOCHS	3	验证 Loss 在第 3 轮已开始上升
WEIGHT_DECAY	0.01	AdamW 正则化系数
优化器	AdamW	自适应学习率 + 权重衰减
学习率调度	线性衰减	学习率从 2×10^{-5} 线性衰减到 0
验证集比例	10%	从训练集中随机抽取

3.2 实验结果

表 1：BERT 训练过程

Epoch	训练 Loss	验证 Loss	验证 Acc	备注
1	0.2667	0.2209	91.00%	快速收敛
2	0.1319	0.2391	91.36%	持续提升
3	0.0511	0.2947	91.40%	过拟合信号

最终测试集结果：

指标	数值	评价
Accuracy	92.12%	超过 90% 优秀线
F1-Score	92.11%	与 Accuracy 接近
训练耗时	约 23 分钟	RTX 5060 GPU
误分类数	1,971 / 25,000	错误率 7.88%

3.3 Loss 曲线分析

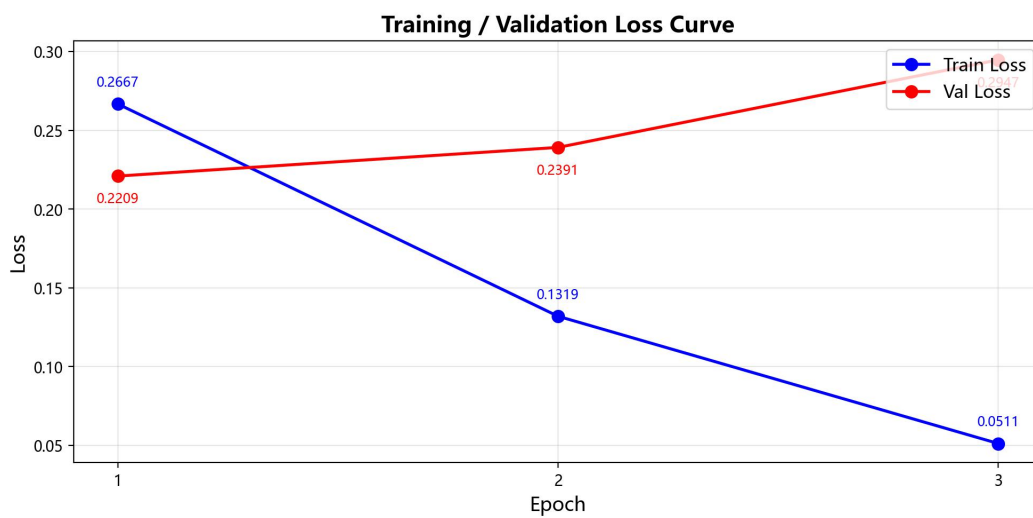


图 1：训练 / 验证损失曲线

从 Loss 曲线可以观察到以下现象：

- (1) 训练 Loss 持续大幅下降 ($0.27 \rightarrow 0.13 \rightarrow 0.05$)，说明模型在训练集上的拟合程度不断提高，优化过程正常收敛。
- (2) 验证 Loss 在第 1 个 Epoch 后出现上升趋势 ($0.22 \rightarrow 0.24 \rightarrow 0.29$)，与训练 Loss 形成典型的「训练-验证背离」现象。这表明从第 2 个 Epoch 开始，模型已经开始出现过拟合——尽管此时验证准确率仍在缓慢上升。
- (3) 验证准确率在 3 个 Epoch 内持续上升 ($91.00\% \rightarrow 91.36\% \rightarrow 91.40\%$)，说明 3 个 Epoch 是合理的训练截止点。若继续训练，验证 Loss 的上升趋势将最终导致验证准确率下降。

Loss 曲线分析是判断过拟合的核心方法——它比 Accuracy 更敏感。在本实验中，验证 Loss 从 Epoch 2 就已经发出了过拟合预警，而验证 Accuracy 的上升趋势掩盖了这一信号。

4 结果分析

4.1 混淆矩阵

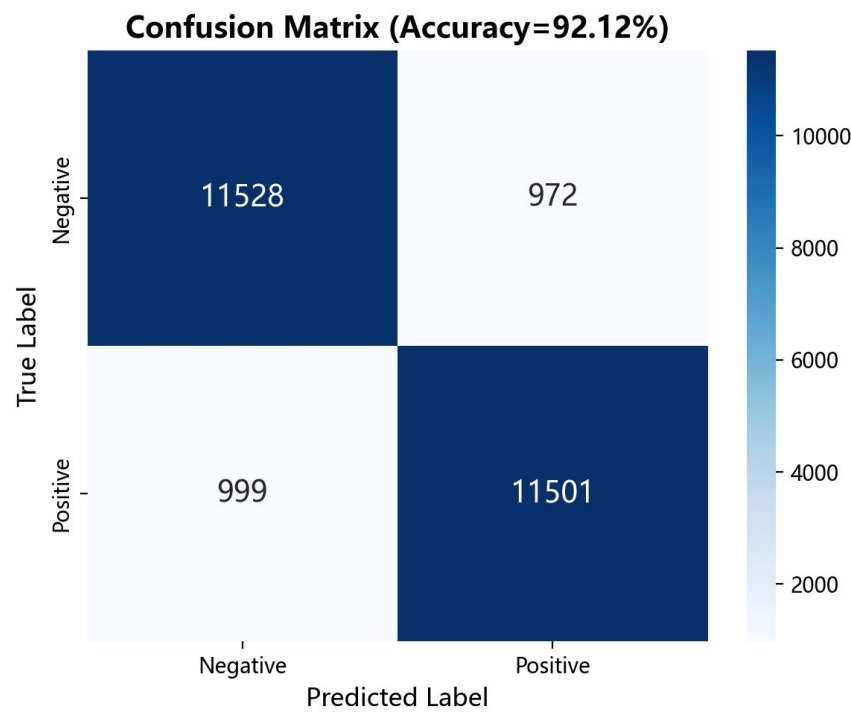


图 2：测试集混淆矩阵

混淆矩阵各象限的含义及本实验数值：

象限	含义	数值	解读
左上 (TN)	真负例：负面 → 负面	约 11,528	识别负面能力强
右下 (TP)	真正例：正面 → 正面	约 11,501	识别正面能力强
右上 (FP)	假正面：负面 → 正面	972	7.88% 错误主要来源之一
左下 (FN)	假负面：正面 → 负面	999	与 FP 几乎相等

两类错误数量几乎相等（972 vs 999，差值仅 0.11%），说明模型没有系统性偏向某一类情感。这是数据正负样本平衡（各占 50%）的自然结果。

5 错误分析

5.1 总体统计

BERT 在 25,000 条测试集上共产生 1,971 条误分类，错误率为 7.88%。其中：

1. 假正面（负面 → 正面）：972 条（49.3%）
2. 假负面（正面 → 负面）：999 条（50.7%）

5.2 错误模式分类

通过对误分类样本的人工阅读和分析，归纳出以下四种典型错误模式：

模式一：转折句式

评论文本前半段使用大量正面词汇进行铺垫，后半段通过 but/however 转折后才表达真实负面评价。

BERT 虽然具备双向注意力机制，但对长距离转折关系的权重分配仍不够精准，容易被前半段的正面信号主导。

【例】*I'm the type of guy who loves hood movies... but this movie was something different.*（真实：负面，预测：正面）——前半段的 *loves* 等正面词导致模型误判

模式二：否定结构

否定词（not, never, wasn't）改变了所在句子的情感倾向。当否定词与关键情感词距离较远时，

BERT 的自注意力机制难以准确将否定信号传导至目标词。

【例】*You probably will NOT like this movie.*（真实：负面，预测：正面）——NOT 虽强调否定，但 *like* 的正面信号仍占据主导

模式三：混合情感（正负交织）

评论同时包含正面和负面评价，模型需要判断整体情感倾向。当正负信号强度相近时，模型倾向于被具体的、显性的正面词汇（如 *enjoy*, *good*, *excellent*）所主导，而忽略负面的整体语境。

【例】*you'll probably enjoy this little flick... Beginning and end are pretty good, middle drags at times.*（真实：负面，预测：正面）——*enjoy* 和 *pretty good* 压过了整体负面评价

模式四：讽刺与含蓄表达

字面上使用正面词汇（*excellent*, *good*, *great*），但实际表达的是讽刺或隐晦的负面评价。这种表达高度依赖文化背景和常识推理，是当前纯文本预训练模型难以突破的瓶颈。

【例】*for an Al Adamson film, it's exceptional* ——表面赞扬（*exceptional* = 极好），实为讽刺（仅在他拍的烂片中算好的）。（真实：负面，预测：正面）

5.3 典型案例

1. 案例 1: *First off let me say, If you haven't enjoyed a Van Damme movie since bloodsport, you probably will not like this movie.* ——前半段隐含对演员的正面期待，后半段否定（*will not like*），模型被前半段的 *enjoyed* 误导。
2. 案例 2: *Low budget horror movie. If you don't raise your expectations too high, you'll probably enjoy this little flick. Beginning and end are pretty good, middle drags at times and seems to go nowhere for long stretches.* ——*enjoy*、*pretty good* 等正面词连续出现，压过了整体的负面判断。
3. 案例 3: *The only reason this movie is not given a 1 (awful) vote is that the acting of both Ida Lupino and Robert Ryan is superb.* ——*superb*（极好）是强烈的正面信号，但整句实际在说除了演员的表演外一无是处。

5.4 改进建议

1. 增大 max_length: 从 256 提升到 512, 减少截断损失, 保留更多评论尾部的关键信息。尤其对于「先扬后抑」模式的评论, 关键否定信息往往出现在评论后半段。
2. 尝试更强的预训练模型: 使用 RoBERTa 或 DeBERTa 替换 BERT-base, 这些模型在更大语料上进行了更充分的预训练, 对否定和讽刺的理解能力更强。
3. 数据增强: 针对转折/否定/讽刺类型的样本进行针对性增强, 例如使用回译 (back-translation) 生成更多含 but/however 结构的训练样本。
4. 对抗训练: 使用 FGM 或 PGD 在训练过程中对 embedding 层施加扰动, 提升模型对语义微小变化的鲁棒性。

6 实验复盘与总结

6.1 调参经验

1. BERT 学习率 2×10^{-5} 是经论文和社区验证的黄金值。本实验验证了其合理性: 训练过程平稳收敛, Loss 持续下降, 未出现震荡或停滞。若从头再来, 可对比尝试 3×10^{-5} 和 1×10^{-5} 。
2. 8 GB 显存条件下, batch_size=16 是稳定运行的上限。使用混合精度训练 (AMP) 理论上可提升至 32, 更大的 batch 可能带来更稳定的梯度估计。
3. 3 个 Epoch 是合理的训练截止点。验证 Loss 在第 3 轮已明显上升, 而验证 Acc 仍在缓慢上升——这是典型的「收益递减但风险递增」阶段, 继续训练边际收益极低。
4. AdamW 的 weight_decay=0.01 提供了一定正则化效果, 但第 3 轮验证 Loss 的上升表明过拟合仍在发生。可尝试增大至 0.05 或增加 Dropout 率。

6.2 代码实现经验

问题	解决方案	经验教训
网络代理无法下载模型和数据	改为本地 parquet + 本地 BERT 加载	离线加载策略是应对网络不稳定环境的必要技能
matplotlib 中文标题显示为方块	配置中文字体 + unicode_minus=False	Windows 下最常见的中文显示问题
Git 推送大文件失败 (3.5 GiB)	创建 .gitignore 排除 venv/Data/模型	大文件必须在仓库外管理
虚拟环境 pip 包缺失	激活 venv 后重新安装全部依赖	虚拟环境是空白的，系统包不会自动带入
错误分析需要原始评论文本	保留 dataset 原始引用，不过早 remove_columns	数据预处理需同时考虑训练和分析需求

6.3 关键收获

1. 第一，「预训练+微调」范式极大降低了 NLP 任务的门槛。只需在 BERT 上添加一个简单的分类头，用 3 个 Epoch、23 分钟的训练，就能达到 92.12% 的准确率。这在 5 年前是不可想象的——当时需要从零训练 LSTM，耗时数小时且准确率仅约 87%。
2. 第二，Loss 曲线的「训练-验证背离」是判断过拟合的核心信号，比 Accuracy 更敏感。本实验中验证 Loss 从第 2 个 Epoch 就开始预警，而验证 Accuracy 直到实验结束仍在上升——如果只看 Accuracy，很可能会误判并继续训练，导致更严重的过拟合。
3. 第三，单一的 Accuracy 指标远远不够。92.12% 看起来出色，但混淆矩阵和错误分析揭示：仍有 1,971 条评论被误判，且错误高度集中在转折、否定、混合情感、讽刺等特定语言模式上。理解模型在哪里犯错比知道错了多少更有价值。

4. 第四，情感分析的核心挑战不在于识别正面词汇，而在于理解否定、转折、讽刺等需要深层语义推理的语言现象。这些现象恰恰是 BoW 等传统方法完全无法处理的，也是 BERT 等预训练模型仍需改进的方向。

7 AI 工具使用说明

本次实验中使用了 Claude (Anthropic) 进行以下辅助：

1. 代码讲解：train.py / evaluate.py / inference.py 中每行代码的作用和底层原理
2. 环境配置：CUDA 版本选择、虚拟环境使用、pip 安装命令指导
3. 故障排查：缩进修复、代理网络问题诊断、Git 大文件推送失败处理
4. 报告撰写：实验报告的文字组织、格式排版和表达润色

所有代码的核心逻辑已经过本人理解和验证，可以独立解释每一部分的实现原理。报告中的错误案例分析由本人阅读误分类样本后归纳总结，AI 仅用于辅助文字组织。